

Task delivered. Where did the budget go?

Understand how agents spend tokens, compute and time. Find the bottleneck and check whether the result was worth the resources.

PATH TO DELIVERY

01 → 04

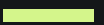
01 Understand the task



02 Make the changes



03 Wait for the test agent



↺ Check again



04 Deliver the result



Illustrative workflow. Bar lengths do not represent measured shares or durations.

The total hides the bottleneck.

A token summary tells us how much was used. People need to know which step is slowing delivery and what they can change.

01

Expensive waiting

The lead agent keeps calling its model while waiting for a smaller agent to finish.

02

Checks on repeat

Tests run again. We need to see whether a change, failure or new requirement justified the next run.

03

Spend without an outcome

High resource use alone says little about success. The task may still be unfinished.

OUR QUESTION What useful outcome did we pay for?

The lead agent returns to wait.

In one session, nearly half of the lead agent's recorded tokens belong to model responses that invoke waiting.

The same session delegates tests to `full_tests_luna`. The lead also waits for other agents, so these tokens cannot all be attributed to tests.

46.5%

of lead-agent tokens are in responses invoking
`wait_agent / sleep`



`wait_agent` calls

571

Wait-associated tokens

59.2M

From Codex logs Input + output. 98.7% of input in these responses is cached. Token share is not cost share. The wait itself does not generate model tokens.

One test observer. Hundreds of polls.

Even a smaller agent can accumulate substantial usage by frequently checking a process and reporting back to the lead.

505

process polls

`write_stdin` calls in the test session.
Polling a running process is a separate action from launching tests.

386

messages to the lead

`send_message` calls. Separate model steps add context and consume tokens.

4

package test runs

`go test`
`./internal/adapters/reconcileruntime` in one session. Test filters differed; the need for each run requires review.

PATTERNS TO DETECT · Frequent polling · redundant updates · reruns without a new reason

Spot the pattern. Explain the cause.

SIGNAL	EVIDENCE TO CHECK	POSSIBLE CHANGE
Waiting with an expensive model	Repeated wait-invoking responses, agent role, context size and cached input.	Use completion events. Do independent work while tests run.
A test already passed	The same check, code and environment, with a previous passing result.	Reuse the result when the relevant inputs have not changed.
Too much observation	Polling and update frequency, new information between calls, time to delivery.	Increase the interval. Report state changes and completion.

A signal starts a review. A rerun after a code change, failure or flaky test may be justified.

One step. Several dimensions.

T

Tokens

Input, cached input and output separately. Model, agent role and specific step.

\$

Cost

Model pricing on the request date. Keep estimates separate from actual billing.

↻

Compute

CPU, memory and test processes. Process elapsed time separately from CPU time.

→

Delivery

Time to outcome, waits and blockers. Task success and the checks that confirm it.

Task → Agent → Model step / tool → Usage + outcome

Cached tokens are part of input. Reasoning tokens are part of output. Avoid double counting and keep tokens separate from dollars.

Start with Subyard.

Reach other workflows.

AVAILABLE IN SUBYARD

Logs and usage

AI Observer collects Codex and Claude Code sessions. yard usage forwards requests to ccusage.

A foundation for investigation

PROJECT GOAL

Explain resource use

Connect steps to tokens, processes, time and outcomes. Show where the bottleneck formed.

Validate with real sessions

PROJECT GOAL

Portable collection

Standard devcontainers and other environments: Codex, Claude, OpenCode. Common metrics through adapters.

Without requiring Subyard

Subyard's current AI Observer file integration does not collect OpenCode or pi. Universal collection is a project requirement, not a shipped feature.

Less guesswork. More useful work.

People should quickly see where the budget goes, why delivery is delayed and which change could improve the next run.

01 What was delivered? 02 Where did resources go? 03 What should change?